Exact Scheduling of a Single Additive Manufacturing Machine for Total Tardiness Minimization

***Abstract*— This paper studies the single-machine one-dimensional batch scheduling problem for additive manufacturing with the objective of minimizing total tardiness. To solve this problem exactly, we develop a tailored branch-and-bound (B&B) algorithm that integrates hybrid lower-bounding, adaptive node selection, and dominance-based pruning mechanisms. Computational experiments show that the proposed B&B clearly outperforms Gurobi in both efficiency and scalability. The results further reveal opposite computational effects of platform capacity: a smaller platform makes the mixed-integer linear program formulation harder for Gurobi, while improving the efficiency of the proposed B&B algorithm.**

I. INTRODUCTION

As Additive Manufacturing (AM) evolves from a rapid prototyping tool into a practical technology for end-use production, it has become increasingly important in customized manufacturing and emerging social manufacturing settings [1][2]. In these settings, customer orders are often highly personalized and time-sensitive, which makes due-date compliance a critical operational concern. Consequently, efficient production scheduling is essential for ensuring timely order fulfillment.

Existing AM scheduling research mainly focuses on heuristic and metaheuristic methods. Although these approaches are often computationally efficient for large-scale instances, they do not provide optimality guarantees. In highly customized production settings with strict delivery requirements, exact algorithms remain valuable because they can provide optimal solutions for small- and medium-sized instances and serve as benchmarks for heuristic methods.

Among the existing exact approaches for AM scheduling, Mao et al. [3] studied the single-machine batch scheduling problem and proposed a Combinatorial Benders Decomposition (CBD) algorithm for makespan minimization. Similarly, Zipfel et al. [4] developed a branch-and-cut approach for the parallel-machine scheduling problem, also with the objective of minimizing makespan. However, these studies primarily focus on production throughput and do not explicitly address due-date-related objectives.

To the best of our knowledge, the only exact approaches for minimizing total tardiness are due to Nascimento et al. [5], who applied CBD to a parallel-machine scheduling problem and later improved the solution procedure through Logic-based Benders Decomposition (LBBD) [6]. Nevertheless, these approaches model the problem as a standard p-batch scheduling problem. In such a setting, the processing time of a batch is determined solely by the maximum processing time among its constituent jobs. In contrast, in prevalent AM processes such as Powder Bed Fusion, the actual batch processing time depends on both the maximum part height and the total part volume in the batch. This feature makes the problem fundamentally different and places it in the class of M-batch scheduling problems.

Our preliminary investigation shows that directly extending CBD or LBBD to the single-machine AM M-batch tardiness problem leads to serious computational difficulties. The main reason is that the mathematical programming model of the master problem, i.e., one-dimensional batch scheduling, becomes much harder to solve when height-dependent processing times are considered. Since Benders-type methods require the master problem to be solved repeatedly during convergence, this modeling complexity becomes a major computational bottleneck, severely limiting scalability and making it difficult to obtain global optimal solutions within a reasonable time.

To address this challenge, this paper develops an exact method for minimizing the total tardiness of a single AM machine. The main contributions are summarized as follows:

- We propose a branch-and-bound (B&B) algorithm to solve the problem exactly. To improve computational efficiency, the proposed method incorporates multiple node selection strategies, problem-tailored upper and lower bounds, and dominance-based pruning rules.
- Computational experiments show that the proposed B&B algorithm outperforms Gurobi and further confirm the effectiveness of the hybrid lower-bounding framework and the proposed acceleration strategies.

The remainder of this paper is organized as follows. Section II presents the problem description and mathematical formulation. Section III introduces the proposed B&B algorithm. Section IV reports the computational results. Finally, Section V concludes the paper.

II. MATHEMATICAL PROGRAMMING MODEL

A. Problem Description

A set of parts $J = \{1, \ldots, n\}$ is to be processed on a single metal additive manufacturing machine based on Powder Bed Fusion technology. Without loss of generality, we assume that each customer order consists of a single part, so that order tardiness is equivalent to part tardiness.

This paragraph of the first footnote will contain the author affiliations, contact information, and email addresses in the final camera-ready version. It is currently blinded to comply with double-anonymous review guidelines.

This paragraph of the second footnote will contain funding information and grant numbers in the final camera-ready version. It is currently blinded to comply with double-anonymous review guidelines.

Each part $j \in J$ has a due date d_j, a volume v_j, and fixed geometric dimensions given by length l_j, width w_j, and height h_j. The build platform is rectangular, with length L and width W.

Parts are grouped into batches and processed sequentially on the machine. We consider a one-dimensional capacity restriction, under which a set of parts can be assigned to a batch if the sum of their projected areas does not exceed the platform capacity $L \times W$. This assumption gives rise to the one-dimensional batch scheduling problem studied in this paper. The processing time of a batch b, denoted by P_b, consists of three components: the fixed machine setup time S, the laser scanning time, and the recoating time [7]. Let C_b denote the completion time of batch b. If part j is assigned to batch b, then its completion time c_j is equal to C_b. The tardiness of part j is defined as $T_j = \max\{0,\, c_j - d_j\}$. All parts are assumed to be available at time zero. The objective is to determine the assignment of parts to batches and the sequence of batches on the machine so as to minimize the total tardiness.

B. MILP Formulation

We formulate the single-machine one-dimensional batch scheduling problem as a mixed-integer linear program (MILP). The model is adapted from Mao et al. [3], who studied a single-machine AM scheduling problem with a makespan objective under two-dimensional packing constraints. Compared with their formulation, two modifications are introduced here. First, the objective is changed from makespan minimization to total tardiness minimization. Second, the original two-dimensional packing constraints are replaced by the one-dimensional capacity restriction adopted in this paper, under which the feasibility of a batch is approximated by the total projected area of its assigned parts.

Sets and Indices

- $J = \{1, \ldots, n\}$: Set of parts.
- $B = \{1, \ldots, n\}$: Set of available batches on machine.

Parameters

- l_j, w_j, h_j: Length, width, and height of part j.
- v_j: Volume of part j.
- d_j: Due date of part j.
- L, W: Length and width of the build platform.
- S: Setup time of machine.
- V: Laser scanning time of machine.
- U: Recoating speed of machine.
- M: A sufficiently large positive constant.

Decision Variables

- x_{jb}: equal to 1 if part j is assigned to batch b, 0 otherwise.
- z_b: equal to 1 if batch b is used, 0 otherwise.
- H_b: height of batch b.
- P_b: processing time of batch b.
- C_b: completion time of batch b.
- c_j: completion time of part j.
- T_j: tardiness of part j.

The MILP formulation is given as follows:

$$\text{Minimize} \quad \sum_{j \in J} T_j \tag{1}$$

s.t.

$$\sum_{b \in B} x_{jb} = 1, \quad \forall j \in J \tag{2}$$

$$\sum_{j \in J} x_{jb} \le M z_b, \quad \forall b \in B \tag{3}$$

$$\sum_{j \in J} (l_j \cdot w_j) \cdot x_{jb} \le LW, \quad \forall b \in B \tag{4}$$

$$H_b \ge h_j \cdot x_{jb}, \quad \forall j \in J, \forall b \in B \tag{5}$$

$$P_b = S z_b + V \sum_{j \in J} (v_j \cdot x_{jb}) + U H_b, \quad \forall b \in B \tag{6}$$

$$C_b \ge C_{b-1} + P_b, \quad \forall b = 2, \ldots, |B| \tag{7}$$

$$c_j \ge C_b - M(1 - x_{jb}), \quad \forall j \in J, \forall b \in B \tag{8}$$

$$T_j \ge c_j - d_j, \quad \forall j \in J \tag{9}$$

$$T_j \ge 0, \quad \forall j \in J \tag{10}$$

$$z_{b+1} \le z_b, \quad \forall b = 1, \ldots, |B| - 1 \tag{11}$$

$$x_{jb}, z_b \in \{0, 1\}, \quad H_b, P_b, C_b, c_j, T_j \ge 0 \tag{12}$$

The objective function (1) minimizes the total tardiness of all parts. Constraints (2) ensure that every part is assigned to only one batch. Constraints (3) ensure that batch b is opened if at least one part is allocated to this batch. Constraints (4) enforce the capacity constraint of the build platform, ensuring that the total projection area of parts in a batch does not exceed the platform's surface area. Constraints (5) determine the height of batch b as the maximum height of the assigned parts. Constraints (6) calculate the processing time of each batch. Constraints (7) calculate the batch completion times. Constraint (8) determines the completion time of individual parts based on their assigned. Constraints (9) and (10) define the tardiness of each part. Finally, Constraints (11) enforce a contiguous batch usage order (i.e., batch $b + 1$ cannot be used unless batch b is used) to break symmetry and improve computational efficiency.

III. METHODOLOGY

This section presents an exact B&B algorithm for the single-machine AM batch scheduling problem. The algorithm follows the general search framework of Tangudu and Kurz [8], while being adapted to the structure of the problem studied here. We first describe the overall search framework, including the branching scheme for node generation and the node selection strategy for tree exploration. We then introduce the upper bound(UB), the lower bound(LB), and the dominance rules used to prune the search space and improve computational efficiency.

A. Branch-and-Bound Algorithm Structure

The proposed B&B algorithm explores the solution space of the single-machine AM batch scheduling problem in a systematic manner. A fast constructive heuristic is first

applied to obtain an initial feasible solution, which provides the initial global UB. The search is then initialized at the root node, representing an empty schedule.

At each iteration, the current active node is branched by selecting a feasible subset of the unscheduled parts to form the next batch, while respecting the build-platform capacity constraints. To control the combinatorial growth of the search tree, dominance rules are applied during node generation to discard redundant child nodes. For each remaining node, a LB is evaluated. A partial node is pruned whenever $LB \geq UB$, whereas a complete node updates the incumbent solution if its objective value improves the current UB.

The surviving nodes remain active in the search tree, and the next node is chosen according to a predefined node selection strategy. The algorithm terminates when the set of active nodes becomes empty, at which point the incumbent solution is guaranteed to be optimal.

B. Upper Bound Calculations

An initial global UB is obtained by a fast constructive heuristic before the branch-and-bound search begins. The heuristic first sorts all parts in non-decreasing order of due dates according to the earliest-due-date rule. It then constructs a schedule by scanning the parts in this order and assigning them to batches using a greedy first-fit policy. A part is added to the current batch if the build-platform capacity constraint $L \times W$ remains satisfied; otherwise, a new batch is created. The total tardiness of the resulting feasible schedule is used as the initial UB.

C. Node Generation

In the search tree, each node represents a partial schedule composed of a sequence of batches fixed from time zero and a set of parts that remain unscheduled. Since fixing the content of the next batch simultaneously determines its position in the batch sequence, node generation can be viewed as a branching process on the next batch to be appended to the current partial schedule.

A straightforward branching scheme is to generate child nodes by selecting a subset of the unscheduled parts as the next batch. More precisely, let $\mathcal{U}$ denote the set of unscheduled parts at the current node. For any subset $B \subseteq \mathcal{U}$ satisfying the necessary platform-capacity condition

$$\sum_{j \in B} a_j \leq L \times W,$$

where $a_j = l_j w_j$ is the projected area of part j, one may create a child node by appending B to the current partial schedule and updating the unscheduled set to $\mathcal{U} \setminus B$. In this way, each child node corresponds to one possible choice of the next batch.

This branching scheme is conceptually simple, and it directly reflects the decision structure of the problem. However, it may lead to a very large branching factor, since the number of area-feasible subsets of $\mathcal{U}$ can be enormous. Moreover, at the node-generation stage, the above area condition is only a necessary condition for batch formation, while the actual two-dimensional packing feasibility must still be verified separately. As a result, many generated child nodes may eventually be found infeasible or of little value for bounding, which may substantially weaken the efficiency of the search.

To better control the branching factor while preserving exactness, a more structured branching scheme can be adopted by constructing the next batch incrementally. Specifically, in addition to the sequence of closed batches, each node maintains one currently open batch B^{open}, together with the set of parts that have not yet been assigned. At each step, a part j is selected from the remaining set, and two child nodes are generated:

- **Append branch:** part j is appended to the current open batch B^{open}, provided that the necessary area condition remains satisfied;
- **Close-and-open branch:** the current open batch B^{open} is closed and fixed as the next batch in the sequence, and a new open batch is started with part j.

Under this scheme, node generation no longer enumerates all possible next batches explicitly. Instead, the content of the next batch is determined through a sequence of local branching decisions, each of which either expands the current batch or starts a new one. Since closing the current open batch fixes an additional batch boundary, this branching strategy usually provides stronger state progression than a pure membership-based branching rule, and is therefore more favorable for lower-bound evaluation. In particular, it reveals earlier how many batch setups have already occurred and how far the completion times of future parts must be shifted, which is beneficial for the proposed tardiness lower bounds.

To avoid generating duplicate representations of the same batch, the unassigned parts are processed according to a fixed order. Therefore, each feasible complete batch sequence is represented by at least one path in the search tree, while redundant symmetric constructions are reduced.

D. Node Selection

To explore the search tree efficiently, three node selection strategies are considered: Best-First Search (BFS), Depth-First Search (DFS), and an adaptive hybrid strategy. BFS is intended to improve the global lower bound more quickly by selecting the most promising active node, whereas DFS drives the search deeper into the tree and can help control the number of stored active nodes. To balance convergence behavior and memory usage, an adaptive strategy, denoted by BFS&DFS, is introduced. The strategy maintains an open container for active nodes and uses two thresholds, $N_{\max}$ and $N_{\min}$, where $N_{\max}$ denotes the upper capacity limit and $N_{\min}$ denotes the recovery threshold. The search starts with Best-First Search by default. When the number of active nodes exceeds $N_{\max}$, the strategy switches to DFS to relieve memory pressure. Once the container size decreases below $N_{\min}$, the search switches back to Best-First Search.

E. Lower Bound Calculations

To efficiently prune the search tree, we employ three different lower bound calculate methods.

1) Parallel Lower Bound: The first component is a computationally inexpensive parallel lower bound, denoted by LB_{par}. This bound relaxes the single-machine capacity constraint by assuming that each unscheduled part can be processed independently on its own parallel machine immediately after the current node completion time C_{now}. Hence, the tardiness contribution of each remaining part is evaluated independently rather than through a common batch completion time. Let $\mathcal{U}$ be the set of unscheduled parts, and let $TT_{partial}$ denote the exact total tardiness contributed by the batches already fixed in the current partial schedule. Then, LB_{par} is defined as

$$LB_{par} = TT_{partial} + \sum_{j \in \mathcal{U}} \max(0, C_{now} + S + V \cdot v_j + U \cdot h_j - d_j) \tag{13}$$

2) Serial Lower Bound: Although LB_{par} is fast to evaluate, it ignores the sequential nature of the remaining scheduling decisions. To obtain a tighter bound, we construct a serial lower bound, denoted by LB_{ser}. Consider any feasible continuation of the current partial schedule. Such a continuation partitions the unscheduled set $\mathcal{U}$ into m batches, denoted by $B_1, B_2, \ldots, B_m$, which are processed sequentially after time C_{now}. For a part $j \in B_k$, its completion time in the corresponding batch schedule is

$$C_j^{\text{real}} = C_{now} + \sum_{r=1}^{k} \left(S + V \sum_{i \in B_r} v_i + U \max_{i \in B_r} h_i \right). \tag{14}$$

To derive a tractable relaxation, we map the above batch schedule to an induced sequence π obtained by listing the parts batch by batch in the order $B_1, B_2, \ldots, B_m$. Let $h_{\min} := \min_{j \in \mathcal{U}} h_j$. We then define a relaxed single-machine total tardiness problem in which the setup term S and the height-dependent term $Uh_{\min}$ are charged only once for the entire unscheduled set, while the volume-dependent term accumulates part by part along the induced sequence. Under this relaxation, the completion time of the part at position p in π is

$$\widetilde{C}_{\pi(p)} = C_{now} + S + Uh_{\min} + V \sum_{r=1}^{p} v_{\pi(r)}. \tag{15}$$

Let $T_{batch}(\pi)$ denote the total tardiness of the original batch schedule associated with π, and let $T_{relax}(\pi)$ denote the total tardiness under the relaxed completion times $\widetilde{C}_{\pi(p)}$. Let π^* be an optimal sequence for the relaxed problem. The following result justifies the bound.

Proposition 1. *For any feasible batch continuation and its induced sequence π,*

$$T_{batch}(\pi) \geq T_{relax}(\pi) \geq T_{relax}(\pi^*). \tag{16}$$

Consequently,

$$\begin{aligned} LB_{ser} &= TT_{partial} + T_{relax}(\pi^*) \\ &= TT_{partial} + \sum_{p=1}^{|\mathcal{U}|} \max\Big(0,\, \widetilde{C}_{\pi^*(p)} - d_{\pi^*(p)}\Big) \end{aligned} \tag{17}$$

is a valid lower bound for the current node.

Proof. Consider any part $j \in \mathcal{U}$ located at position p in the induced sequence π, and suppose that $j \in B_k$. The actual completion time C_j^{real} contains at least one setup term and at least one height-dependent term, whereas the relaxation charges only $S + Uh_{\min}$ once. Moreover, the cumulative processed volume up to batch B_k in the batch schedule is at least the cumulative volume of the first p parts in the induced sequence. Therefore,

$$C_j^{\text{real}} \geq \widetilde{C}_{\pi(p)}, \qquad \forall j \in \mathcal{U}.$$

Since tardiness is nondecreasing in completion time, this implies

$$T_{batch}(\pi) \geq T_{relax}(\pi).$$

The second inequality follows directly from the optimality of π^* for the relaxed problem. Hence,

$$T_{batch}(\pi) + TT_{partial} \geq T_{relax}(\pi^*) + TT_{partial} = LB_{ser},$$

which proves the claim. □

The relaxed problem is equivalent to a $1||\sum T_j$ scheduling problem for the parts in $\mathcal{U}$, where part j has processing time Vv_j and all jobs share the same release offset $C_{now} + S + Uh_{\min}$. We solve this problem by Lawler's dynamic programming algorithm [9]. To improve efficiency within the branch-and-bound procedure, a global caching mechanism is incorporated to reuse previously computed states across different nodes.

3) Positional Lower Bound: Although LB_{ser} captures the cumulative volume contribution of the remaining parts along a serial sequence, it remains conservative because the setup term and the height-dependent term are charged only once for the entire unscheduled set. To better reflect the batch structure of the remaining decisions while preserving computational efficiency, we further derive a positional lower bound, denoted by LB_{pos}.

The key idea is to directly lower bound the completion time of the k-th completed part among the unscheduled parts. Under the total tardiness objective, this position-wise view is more appropriate than bounding future batch completion epochs only at the batch level.

Let $\mathcal{U}$ be the set of unscheduled parts at the current node, and let $m := |\mathcal{U}|$. For each part $j \in \mathcal{U}$, let

$$a_j := l_j w_j$$

denote its projected area, and let h_j and v_j denote its height and volume, respectively.

To construct a lower bound on the completion time of the k-th completed unscheduled part, we introduce three sorted sequences:

$$\begin{aligned} a_{(1)} \leq a_{(2)} \leq \cdots \leq a_{(m)}, \\ h_{[1]} \leq h_{[2]} \leq \cdots \leq h_{[m]}, \\ v_{\langle 1 \rangle} \leq v_{\langle 2 \rangle} \leq \cdots \leq v_{\langle m \rangle}, \end{aligned}$$

where $(\cdot)$, $[\cdot]$, and $\langle \cdot \rangle$ denote the indices after sorting the unscheduled parts by area, height, and volume, respectively.

For each position $k = 1, \dots, m$, define

$$A_k := \sum_{q=1}^{k} a_{(q)}, \qquad \beta_k := \left\lceil \frac{A_k}{LW} \right\rceil,$$

where $L \times W$ is the platform area. Since the first k completed unscheduled parts must occupy total area at least A_k, at least β_k future batches are required before the k-th completion can occur.

Next, define the minimum possible cumulative volume contribution of the first k completed unscheduled parts as

$$\underline{V}_k := \sum_{q=1}^{k} v_{\langle q \rangle}.$$

The remaining difficulty lies in lower bounding the height-dependent contribution of these necessary future batches. To this end, consider the following auxiliary batching problem, which minimizes the sum of batch maximum heights over a feasible partition of k parts:

$$\begin{aligned}
H_k^\star := \min \quad & \sum_{b=1}^{r} H_b && && \text{(18a)}\\
\text{s.t.} \quad & B_b \subseteq \{1, \dots, k\}, && b = 1, \dots, r, && \text{(18b)}\\
& \bigcup_{b=1}^{r} B_b = \{1, \dots, k\}, && && \text{(18c)}\\
& B_b \cap B_{b'} = \emptyset, && 1 \le b < b' \le r, && \text{(18d)}\\
& B_b \neq \emptyset, && b = 1, \dots, r, && \text{(18e)}\\
& \sum_{j \in B_b} a_j \le LW, && b = 1, \dots, r, && \text{(18f)}\\
& H_b = \max_{j \in B_b} h_j, && b = 1, \dots, r, && \text{(18g)}\\
& r \ge \beta_k. && && \text{(18h)}
\end{aligned}$$

Problem (18) captures the minimum possible height-dependent contribution before the k-th completion position. However, it is itself a combinatorial batching problem. To derive a computationally cheap yet valid lower bound, we relax (18) in two steps: (i) the number of batches is fixed to its minimum possible value β_k, and (ii) the capacity constraints (18f) are removed. The resulting relaxation is

$$\begin{aligned}
\widetilde{H}_k := \min \quad & \sum_{b=1}^{\beta_k} H_b && && \text{(19a)}\\
\text{s.t.} \quad & G_b \subseteq \{1, \dots, k\}, && b = 1, \dots, \beta_k, && \text{(19b)}\\
& \bigcup_{b=1}^{\beta_k} G_b = \{1, \dots, k\}, && && \text{(19c)}\\
& G_b \cap G_{b'} = \emptyset, && 1 \le b < b' \le \beta_k, && \text{(19d)}\\
& G_b \neq \emptyset, && b = 1, \dots, \beta_k, && \text{(19e)}\\
& H_b = \max_{j \in G_b} h_{[j]}, && b = 1, \dots, \beta_k. && \text{(19f)}
\end{aligned}$$

By construction, (19) is a relaxation of (18). Hence,

$$\widetilde{H}_k \le H_k^\star.$$

Moreover, the relaxation (19) admits a closed-form optimal solution.

Lemma 1. *For each $k = 1, \dots, m$, the optimal value of* (19) *is*

$$\widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

Proof. Since the β_k groups in (19) must be nonempty, each group contributes the height of at least one assigned part through its group maximum. To minimize the sum of group maxima, $\beta_k - 1$ groups should be made as small as possible, namely singleton groups containing the $\beta_k - 1$ smallest heights $h_{[1]}, \dots, h_{[\beta_k - 1]}$, while all remaining parts are assigned to the last group. The maximum height of that group is then $h_{[k]}$. Therefore,

$$\widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

□

We therefore define

$$\underline{H}_k := \widetilde{H}_k = \sum_{r=1}^{\beta_k - 1} h_{[r]} + h_{[k]}.$$

Based on the above components, define

$$\underline{C}_k := C_{now} + \beta_k S + U \underline{H}_k + V \underline{V}_k, \qquad k = 1, \dots, m.$$

The quantity $\underline{C}_k$ represents a lower bound on the earliest possible completion time of the k-th completed part among the unscheduled set.

Proposition 2. *Let $C_{(k)}^{real}$ denote the completion time of the k-th completed part among the unscheduled parts in any feasible continuation of the current partial schedule. Then, for every $k = 1, \dots, m$,*

$$C_{(k)}^{real} \ge \underline{C}_k.$$

Proof. Fix any $k \in \{1, \dots, m\}$ and consider the first k unscheduled parts completed in an arbitrary feasible continuation.

First, these k parts must occupy total projected area at least

$$A_k = \sum_{q=1}^{k} a_{(q)}.$$

Since each batch can accommodate at most LW units of projected area, at least

$$\beta_k = \left\lceil \frac{A_k}{LW} \right\rceil$$

future batches are required before the k-th completion occurs. Hence, the cumulative setup-time contribution is at least $\beta_k S$.

Second, the total processed volume before the k-th completion must be at least the sum of the k smallest part volumes among $\mathcal{U}$, namely

$$\underline{V}_k = \sum_{q=1}^{k} v_{\langle q \rangle}.$$

Therefore, the cumulative volume-dependent contribution is at least $V\underline{V}_k$.

Third, the cumulative height-dependent contribution before the k-th completion is at least $UH_k^\star$. Since

$$H_k^\star \geq \widetilde{H}_k = \underline{H}_k,$$

it is further bounded below by $U\underline{H}_k$.

Combining the setup, height, and volume contributions yields

$$C_{(k)}^{real} \geq C_{now} + \beta_k S + U\underline{H}_k + V\underline{V}_k = \underline{C}_k.$$

This proves the claim. □

To convert these positional completion-time bounds into a tardiness lower bound, let

$$d_{[1]}^{\uparrow} \leq d_{[2]}^{\uparrow} \leq \cdots \leq d_{[m]}^{\uparrow}$$

be the due dates of the unscheduled parts sorted in non-decreasing order. Then, by a standard pairwise interchange argument, the minimum possible tardiness implied by the completion-time lower bounds $\underline{C}_1, \ldots, \underline{C}_m$ is attained by matching smaller completion-time lower bounds with smaller due dates. Therefore,

$$LB_{pos} = TT_{partial} + \sum_{k=1}^{m} \max\left\{0,\, \underline{C}_k - d_{[k]}^{\uparrow}\right\} \tag{20}$$

is a valid lower bound for the current node.

Compared with LB_{ser}, the bound LB_{pos} explicitly captures the fact that the first k completed unscheduled parts must already consume enough platform area to require multiple future batches. Meanwhile, the height-dependent contribution is obtained through a relaxation of an underlying batching problem, which provides a stronger structural interpretation than a direct ad hoc construction while remaining computationally inexpensive.

4) Hybrid Lower Bound: Although LB_{par} is computationally efficient, it is relatively loose, whereas LB_{ser} provides tighter pruning at the cost of solving an expensive dynamic programming subproblem. To balance computational effort and pruning strength, we adopt a hybrid lower-bounding strategy, denoted by LB_{hyb}.

For each node, the algorithm first evaluates the fast bound LB_{par}. The more expensive bound LB_{ser} is computed only when two conditions are simultaneously satisfied: (1) the relative gap between the global UB and LB_{par} is sufficiently small, suggesting that a tighter bound may lead to pruning; and (2) the proportion of remaining unscheduled parts is sufficiently large, so that pruning at this stage can eliminate a substantial portion of the downstream search space.

Let $|J|$ denote the total number of parts. The hybrid lower bound is computed as

$$LB_{hyb} = \begin{cases} \max(LB_{par}, LB_{ser}), & \text{if } \dfrac{UB - LB_{par}}{UB} \leq 0.10 \\ & \text{and } \dfrac{|\mathcal{U}|}{|J|} > 0.60, \\ LB_{par}, & \text{otherwise.} \end{cases} \tag{21}$$

In this way, the expensive serial bound is evaluated only for nodes where it is expected to provide the greatest pruning benefit.

F. Dominance Property

To reduce the search space, a dominance property is used to eliminate redundant partial schedules. Each node u is characterized by the tuple $(\mathcal{S}_u, C_u, TT_u)$, where $\mathcal{S}_u$ is the set of already scheduled parts, C_u is the completion time of the last scheduled batch, and TT_u is the cumulative total tardiness incurred so far.

Proposition 3. *Node u dominates node v (denoted by $u \prec v$) if $\mathcal{S}_u = \mathcal{S}_v$ and*

$$TT_u \leq TT_v \qquad \textit{and} \qquad C_u \leq C_v \tag{22}$$

with at least one inequality being strict.

Proof. Since $\mathcal{S}_u = \mathcal{S}_v$, the two nodes have the same unscheduled parts and therefore admit the same feasible continuations. For any common continuation, the appended batch processing times are identical. Hence, $C_u \leq C_v$ implies no later future completion times from node u than from node v. Combined with $TT_u \leq TT_v$, this yields no larger total tardiness for any completion of node u. Therefore, node v is dominated and can be pruned. □

IV. NUMERICAL EXPERIMENTS

The computational experiments were designed with four objectives. First, we examine the impact of different node selection strategies on search efficiency. Second, we assess the effectiveness of the lower-bounding strategies through a comparison of the three B&B variants. Third, we evaluate the overall exact-solution performance of the proposed method by comparing BB_{hyb} with Gurobi. Fourth, we investigate the sensitivity of algorithmic performance to different build-platform sizes.

The B&B algorithms were implemented in C++, and the mathematical models are solved by Gurobi Optimizer. All experiments were conducted on a personal computer with an Intel Core i5-14400F processor (2.50 GHz) and 16 GB of RAM. A maximum time limit of 3600 seconds was imposed on each test instance.

A. Test Instances

The computational experiments are conducted on test instances adopted from Mao et al. [3]. For each part j, the due date d_j is generated following the procedure of Yu et

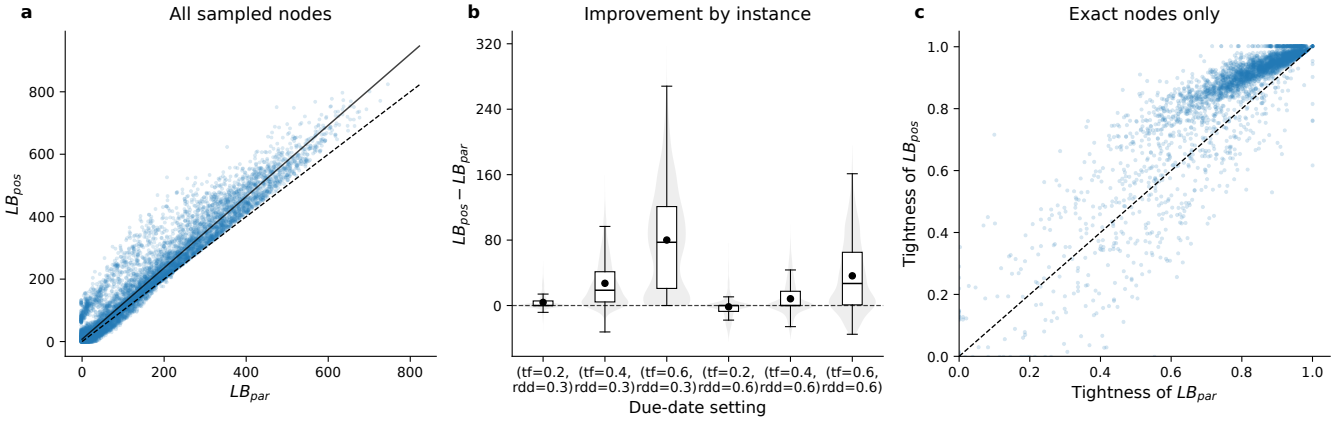

Fig. 1. Comparison between the parallel lower bound LB_{par} and the proposed positional lower bound LB_{pos} under six due-date settings. (a) Node-wise comparison of the two lower bounds over all sampled nodes. (b) Distribution of the improvement $LB_{pos} - LB_{par}$ for different (TF, RDD) settings. (c) Comparison of tightness ratios on nodes for which the exact optimal remaining tardiness can be computed.

TABLE I

COMPARISON OF DIFFERENT NODE SELECTION STRATEGIES

Instance (n)	BFS		DFS		BFS&DFS	
	TT	t (s)	TT	t (s)	TT	t (s)
10	116.30	0.13	116.30	0.16	116.30	0.15
11	110.61	0.45	110.61	0.61	110.61	0.54
12	139.08	1.64	139.08	2.02	139.08	1.86
13	168.72	7.25	168.72	8.66	168.72	7.12
14	156.52	18.18	156.52	24.56	156.52	22.09
15	139.06	64.31	139.06	68.47	139.06	59.14
16	227.70	127.65	227.70	189.81	227.70	176.42
17	237.04	421.07	237.04	639.84	237.04	669.21
18	154.82	1151.59	154.82	1766.18	154.82	1820.25
19	812.62	3600.00	316.82	3600.00	316.82	3600.00

al. [10]. Specifically, due dates are sampled from a uniform distribution:

$$d_j \sim \text{Unif}\big[\mu(1 - TF - RDD/2),\ \mu(1 - TF + RDD/2)\big] \tag{23}$$

where TF and RDD denote the tardiness factor and the relative due-date range, respectively. A larger value of TF corresponds to tighter due dates and therefore a more time-critical demand setting. The parameter μ represents an estimate of the makespan of the instance. For the single-machine setting considered here, μ is estimated by a simple first-fit heuristic in which parts are sorted in non-decreasing order of projected area. In these instances, both part dimensions range from 10 to 19, and the machine platform size is 20×20.

B. Comparison of lower-bound quality

1) Comparison of lower-bound quality: The quality of the proposed positional lower bound LB_{pos} is evaluated by comparing it with the parallel lower bound LB_{par} over a large set of branch-and-bound nodes.

A factorial due-date design is considered with

$$TF \in \{0.2,\ 0.4,\ 0.6\}, \qquad RDD \in \{0.3,\ 0.6\},$$

which yields six due-date scenarios. For each scenario, multiple partial schedules are randomly generated and the corresponding search nodes are collected. At each node, the unscheduled set $\mathcal{U}$ and the current completion time C_{now} are recorded, and both LB_{par} and LB_{pos} are evaluated on the same state. Duplicate nodes with identical $(C_{now}, \mathcal{U})$ are removed. For nodes with small remaining sets, the exact optimal remaining tardiness is additionally computed by dynamic programming, so that the tightness of the two bounds can be assessed against the exact value.

TABLE II

COMPUTATIONAL RESULTS FOR ALGORITHMIC EFFICIENCY (20×20)

Instance			Gurobi				BB_{par}		BB_{ser}		BB_{hyb}	
n	TF	RDD	TT	t (s)	Gap(%)	Stat.	TT	t (s)	TT	t (s)	TT	t (s)
10	0.6	0.6	116.30	2.30	0	Opt.	116.30	0.18	116.30	0.51	116.30	**0.15**
	0.6	0.3	108.79	2.29	0	Opt.	108.79	0.35	108.79	0.39	108.79	**0.15**
	0.3	0.6	37.50	0.41	0	Opt.	37.50	**0.13**	37.50	0.31	37.50	0.13
	0.3	0.3	29.21	0.21	0	Opt.	29.21	**0.10**	29.21	0.23	29.21	0.10
11	0.6	0.6	110.61	14.32	0	Opt.	110.61	0.60	110.61	1.67	110.61	**0.54**
	0.6	0.3	108.35	4.27	0	Opt.	108.35	0.50	108.35	1.24	108.35	**0.47**
	0.3	0.6	38.43	0.38	0	Opt.	38.43	0.24	38.43	0.58	38.43	**0.22**
	0.3	0.3	32.41	0.66	0	Opt.	32.41	0.27	32.41	0.50	32.41	**0.22**
12	0.6	0.6	139.08	53.24	0	Opt.	139.08	2.13	139.08	4.82	139.08	**1.86**
	0.6	0.3	129.26	16.77	0	Opt.	129.26	**1.53**	129.26	3.33	129.26	1.58
	0.3	0.6	41.51	1.45	0	Opt.	41.51	0.68	41.51	1.51	41.51	**0.67**
	0.3	0.3	33.65	2.01	0	Opt.	33.65	**0.56**	33.65	1.16	33.65	0.57
13	0.6	0.6	168.72	410.73	0	Opt.	168.72	8.03	168.72	13.46	168.72	**7.12**
	0.6	0.3	142.01	159.80	0	Opt.	142.01	**3.68**	142.01	7.34	142.01	4.33
	0.3	0.6	42.14	2.44	0	Opt.	42.14	1.66	42.14	3.95	42.14	**1.65**
	0.3	0.3	34.89	4.47	0	Opt.	34.89	**1.64**	34.89	2.52	34.89	1.66
14	0.6	0.6	156.52	553.80	0	Opt.	156.52	**21.40**	156.52	30.96	156.52	22.09
	0.6	0.3	150.23	650.07	0.09	Opt.	150.23	**13.14**	150.23	19.94	150.23	14.06
	0.3	0.6	42.91	12.41	0	Opt.	42.91	**5.27**	42.91	10.46	42.91	5.51
	0.3	0.3	36.43	10.28	0	Opt.	36.43	**5.44**	36.43	7.28	36.43	5.60
15	0.6	0.6	139.06	688.01	0	Opt.	139.06	70.72	139.06	**58.06**	139.06	59.14
	0.6	0.3	132.44	3600.40	53.63	Feas.	129.83	38.92	129.83	**37.31**	129.83	40.18
	0.3	0.6	40.01	46.11	0	Opt.	40.01	**19.91**	40.01	23.35	40.01	20.93
	0.3	0.3	32.31	216.39	0	Opt.	32.31	20.63	32.31	**19.85**	32.31	21.69
16	0.6	0.6	227.70	645.73	0	Opt.	227.70	195.71	227.70	257.09	227.70	**176.42**
	0.6	0.3	206.55	860.76	0	Opt.	206.55	**80.55**	206.55	130.54	206.55	92.62
	0.3	0.6	69.61	**64.38**	0	Opt.	69.61	88.05	69.61	124.23	69.61	72.44
	0.3	0.3	66.03	110.08	0	Opt.	66.03	74.39	66.03	88.83	66.03	**70.78**
17	0.6	0.6	237.04	3600.25	37.00	Feas.	237.04	**650.47**	237.04	796.92	237.04	669.21
	0.6	0.3	214.19	1855.16	0	Opt.	214.19	300.68	214.19	370.93	214.19	**275.86**
	0.3	0.6	72.16	525.71	0	Opt.	72.16	290.27	72.16	382.76	72.16	**246.64**
	0.3	0.3	58.61	513.94	0	Opt.	58.61	232.97	58.61	312.62	58.61	**212.84**
18	0.6	0.6	154.82	3160.25	0	Opt.	154.82	1913.74	154.82	2115.49	154.82	**1820.25**
	0.6	0.3	171.78	2816.55	0	Opt.	171.78	974.22	171.78	**963.10**	171.78	997.36
	0.3	0.6	12.07	**85.09**	0	Opt.	12.07	431.32	12.07	631.61	12.07	399.47
	0.3	0.3	34.07	**233.82**	0	Opt.	34.07	842.36	34.07	883.71	34.07	842.03
19	0.6	0.6	309.87	3600	52.00	Feas.	316.82	3600	316.82	3600	316.82	3600
	0.6	0.3	267.38	3600	48.00	Feas.	401.32	3600	401.32	3600	401.32	3600
	0.3	0.6	103.91	**1554.72**	0	Opt.	301.00	3600	301.00	3600	301.00	3600
	0.3	0.3	78.48	3600	63.00	Feas.	304.07	3600	304.07	3600	304.07	3600

Figure 1 reports the results. In Figure 1a, most points lie above the diagonal, showing that LB_{pos} is larger than LB_{par} on a large majority of sampled nodes. This indicates that the positional bound captures structural information that is absent from the parallel relaxation.

Figure 1b shows the distribution of the improvement $LB_{pos} - LB_{par}$ under different due-date settings. The improvement is nonnegative for most nodes and substantial in several scenarios. In particular, for $RDD = 0.3$, the improvement tends to increase with TF. A similar pattern is also observed for $RDD = 0.6$, although less uniformly. This suggests that the benefit of LB_{pos} depends on the due-date setting, while remaining positive in most cases considered.

Figure 1c compares the tightness ratios of the two lower bounds on nodes for which the exact optimal remaining tardiness is available. Again, most points lie above the diagonal, indicating that LB_{pos} is not only larger than LB_{par}, but also closer to the exact value.

Overall, the results show that LB_{pos} is generally tighter than LB_{par}. This supports its use as a stronger bounding device within the proposed branch-and-bound algorithm.

C. Results analysis

The computational results are reported in Tables I and II. In the table headers, n represents the number of parts, while TF and RDD denote the tardiness factor and the relative due-date range, respectively. The objective value, total tardiness, is denoted by TT, and the computation time in seconds is represented by t (s). For the Gurobi solver, Gap(%) indicates the optimality gap of the best feasible solution, and Stat. reports the final solution status ("Opt." for proven optimality and "Feas." for feasible solutions found within the 3600-second time limit).

a) Impact of node selection strategies.: Table I compares the effect of different node selection strategies for BB_{hyb} under the setting $TF = RDD = 0.6$. While BFS explores the search tree level-by-level and is generally faster overall when solving instances, it delays the discovery of complete schedules (leaf nodes). This results in a lack of effective upper bounds for pruning and carries a high risk of memory explosion. For example, at $n = 19$, BFS times out, leaving only the initial solution ($TT = 812.62$). Conversely, a pure DFS strategy quickly reaches leaf nodes to establish upper bounds with minimal memory overhead, but it blindly dives into unpromising branches without the guidance of tight lower bounds, resulting in lower computational efficiency. The BFS&DFS strategy effectively resolves both flaws. It strictly utilizes early depth-first dives to establish a high-quality global UB (e.g., at $n = 19$, $TT = 316.82$). This bound then prunes the subsequent systematic BFS exploration, successfully avoiding the blind search of DFS while overcoming the memory limitations of BFS, ensuring stability across the instances.

b) Effectiveness of lower-bounding strategies.: Table II compares the overall computational performance of the three B&B variants. Among them, BB_{hyb} shows the most robust overall performance. Across the tested instances, it often achieves the shortest runtime, while in the remaining cases its performance remains close to the best one. By contrast, BB_{par} benefits from very fast bound evaluations but becomes less effective on harder instances, particularly for $n = 17$ and 18, whereas BB_{ser} often incurs additional computational overhead due to repeated dynamic-programming evaluations, despite using a tighter lower bound.

The reason for this behavior is illustrated in Fig. 2, which reports the average number of explored nodes on hard instances. As expected, BB_{ser} explores the fewest nodes, indicating that LB_{ser} provides the strongest pruning effect. In contrast, BB_{par} explores the largest search tree because its weaker bound is less effective in eliminating unpromising nodes. The hybrid strategy BB_{hyb} lies between these two extremes: it retains much of the pruning benefit of LB_{ser} while avoiding the full overhead of repeated dynamic-programming evaluations by activating the serial bound only when stronger pruning is likely to be worthwhile. This explains why BB_{hyb} achieves the best balance between pruning strength and bound-evaluation overhead.

c) Comparison with Gurobi.: As shown in Table II, the proposed B&B algorithms generally outperform Gurobi on the tested instances. For $n \geq 15$, Gurobi frequently reaches the time limit and often leaves substantial optimality gaps, whereas BB_{hyb} is able to solve instances up to $n = 18$. Moreover, tighter due dates noticeably increase the computational burden. For example, at $n = 16$, BB_{hyb} requires 176.42 s under $TF = 0.6$, compared with 70.78 s under $TF = 0.3$. The reason is that tighter due dates make tardiness harder to avoid, thereby reducing the discriminative power of the lower bounds and limiting their pruning effect. Consequently, the algorithm needs to examine more candidate schedules in order to identify and verify the optimal solution.

d) Effect of platform size.: We further examine how platform size affects both scheduling outcomes and computational difficulty. As shown in Fig. 3(a), the restrictive 10×10 platform leads to larger total tardiness values than the more spacious 20×20 platform, because the same set of parts must be distributed over more sequential batches. From a computational perspective, the effect of platform size differs across solution methods, as illustrated in Fig. 3(b). For Gurobi, the smaller platform makes the packing-related constraints tighter, which increases the difficulty of solving the MILP model and leads to longer computational times. For the B&B algorithms, however, the smaller platform reduces the number of feasible part combinations that can form a batch, thereby lowering the branching factor of the search tree. As a result, the B&B algorithms solve the 10×10 instances more efficiently and scale to larger problem sizes than in the 20×20 setting.

V. CONCLUSIONS

This paper studies the single-machine one-dimensional batch scheduling problem in additive manufacturing with the objective of minimizing total tardiness. An exact B&B algorithm is proposed, incorporating upper and lower bounds,

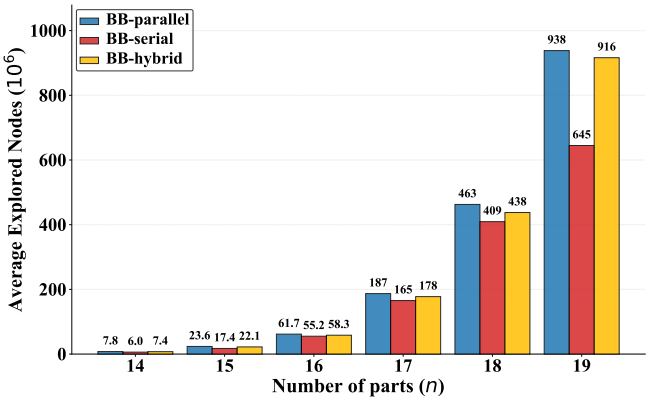

Fig. 2. Comparison of the average explored nodes during tree search.

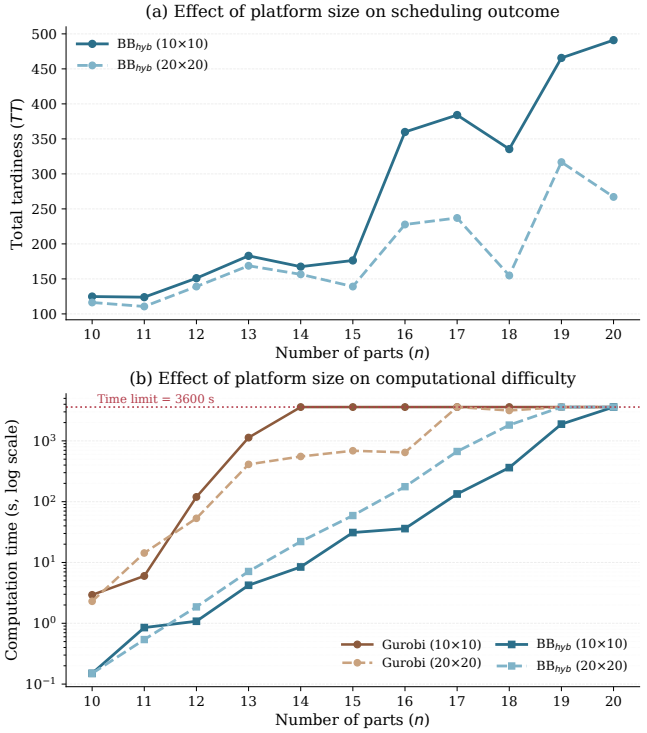

Fig. 3. Comparison of algorithmic performance under different platform sizes ($TF = 0.6$, $RDD = 0.6$).

dominance rules, and multiple node selection strategies. Computational results show that the proposed BB_{hyb} outperforms Gurobi in both efficiency and scalability. The results also reveal an interesting contrast in the effect of platform capacity: a smaller platform increases the difficulty of the MILP formulation for Gurobi, but improves the efficiency of the proposed B&B algorithm. Future work will extend the proposed framework to the single-machine two-dimensional batch scheduling problem with total tardiness minimization.

References

[1] M. Pinto, C. Silva, M. Thürer, and S. Moniz, "Nesting and scheduling optimization of additive manufacturing systems: Mapping the territory," *Computers & Operations Research*, vol. 165, p. 106592, May 2024.

[2] H. Wu, C. Yu, and S. Yu, "Nesting and scheduling for parallel additive manufacturing machines with uncertain processing times: A simulation-optimisation approach," *International Journal of Production Research*, pp. 1–30, Jun. 2025.

[3] Z. Mao, E. Fu, D. Huang, K. Fang, and L. Chen, "Combinatorial Benders decomposition for single machine scheduling in additive manufacturing with two-dimensional packing constraints," *European Journal of Operational Research*, vol. 317, no. 3, pp. 890–905, Sep. 2024.

[4] B. Zipfel, F. Tamke, and L. Kuttner, "A new branch-and-cut approach for integrated planning in additive manufacturing," *European Journal of Operational Research*, p. S0377221724008439, Nov. 2024.

[5] P. J. Nascimento, C. Silva, C. H. Antunes, and S. Moniz, "Optimal decomposition approach for solving large nesting and scheduling problems of additive manufacturing systems," *European Journal of Operational Research*, vol. 317, no. 1, pp. 92–110, Aug. 2024.

[6] P. J. Nascimento, C. Silva, C. H. Antunes, C. T. Maravelias, and S. Moniz, "Improving the efficiency of Logic-based Benders Decomposition for p-batch scheduling problems with two-dimensional packing," *European Journal of Operational Research*, p. S0377221726000767, Jan. 2026.

[7] Y. Che, K. Hu, Z. Zhang, and A. Lim, "Machine scheduling with orientation selection and two-dimensional packing for additive manufacturing," *Computers & Operations Research*, vol. 130, p. 105245, Jun. 2021.

[8] S. K. Tangudu and M. E. Kurz, "A branch and bound algorithm to minimise total weighted tardiness on a single batch processing machine with ready times and incompatible job families," *Production Planning & Control*, vol. 17, no. 7, pp. 728–741, Oct. 2006.

[9] E. L. Lawler, "A "Pseudopolynomial" Algorithm for Sequencing Jobs to Minimize Total Tardiness," in *Annals of Discrete Mathematics*. Elsevier, 1977, vol. 1, pp. 331–342.

[10] C. Yu, A. Matta, Q. Semeraro, and J. Lin, "Mathematical Models for Minimizing Total Tardiness on Parallel Additive Manufacturing Machines," *IFAC-PapersOnLine*, vol. 55, no. 10, pp. 1521–1526, 2022.